

Woxsen University

School of Technology

B.Tech — Computer Science and Engineering

PROJECT BASED LEARNING REPORT

**Market Odyssey — A Student-Run Campus
Marketplace**

*(Design and Implementation of a Full-Stack Web Application for
Buying,
Selling and Swapping Items within the Woxsen Campus, built using
HTML, CSS, PHP, MySQL and JavaScript)*

Course Code: _____
Course Name: Web Technologies
Semester: _____
Academic Year: 2026 – 27

Submitted by:

Name: _____
Register Number: _____
Section: _____

Under the Guidance of:

Faculty Name: _____
Designation: _____

Department of Computer Science and Engineering
September, 2026

DECLARATION

I hereby declare that the project titled “**Market Odyssey — A Student-Run Campus Marketplace**”, submitted in partial fulfilment of the requirements of the course **Web Technologies**, is an original work carried out by me under the guidance of the faculty member named on the title page.

I confirm that this work has not been submitted, in part or in full, for the award of any degree, diploma, or other academic qualification in this or any other institution.

All sources of information used in this report — including code libraries, tutorials, articles, and technical documentation — have been properly acknowledged and cited in the references section, as per the university's referencing standards.

I further declare that the plagiarism percentage in this report is within the permissible limit prescribed by the institution, and that the accompanying source code was written and integrated by me from scratch, using only the topics covered in the Web Technologies syllabus (HTML, CSS, JavaScript, PHP and MySQL), with no external frameworks or build tooling.

Place: Woxsen University

Date: _____

Signature of the Candidate

Name: _____

Register Number: _____

CERTIFICATE

This is to certify that the project titled “**Market Odyssey — A Student-Run Campus Marketplace**” is a bonafide work carried out by **Mr./Ms.** _____
(Register No: _____), student of **B.Tech — Computer Science and Engineering**, during the academic year **2026–27**, in partial fulfilment of the requirements of the course **Web Technologies**, under my supervision and guidance.

The project has been developed using the core technologies of the syllabus — HTML, CSS, JavaScript on the client side and PHP with MySQL on the server side — on the XAMPP development stack, without any external framework or build system.

To the best of my knowledge, the work reported herein does not form part of any other project or dissertation on the basis of which a degree or an award has been conferred on an earlier occasion by any other candidate.

Place: Woxsen University

Date: _____

Guide Signature

Name of the Guide:

Designation: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the faculty of the **Department of Computer Science and Engineering, Woxsen University**, for their consistent encouragement and guidance throughout the course of this project.

I am particularly thankful to my project guide, whose feedback shaped the project from a rough idea into a working, end-to-end web application. Their emphasis on writing clean, syllabus-aligned code — rather than relying on frameworks — forced me to understand every layer of the stack, and that discipline is reflected throughout this report.

I am also grateful to my classmates who acted as early users, tested the buy/sell flow, filed bug reports and offered honest feedback on the interface. Their participation directly influenced the final shape of the reservation flow, the mutual review system and the admin approval workflow.

Finally, I thank the Woxsen student community — the "Buy n Sell" WhatsApp group members whose day-to-day frustration became the motivation for this project.

— The Candidate

ABSTRACT

Market Odyssey is a web-based, student-run campus marketplace built for Woxsen University as part of the Web Technologies course. The application replaces the ad-hoc "Buy n Sell" WhatsApp group — where product photos and prices scroll past faster than any student can track — with a structured, searchable marketplace where every listing is admin-approved before it goes live.

The system implements the full trade lifecycle end-to-end: a seller posts photos, a title, a price, a campus pickup spot and an optional UPI QR code; an administrator reviews and approves the listing; buyers reserve items for a 20-minute window, chat with the seller through an in-app message thread, mutually confirm delivery, and leave a two-sided review after the transaction closes. Sellers can freeze a reservation so the countdown never expires, which is useful when delivery is delayed by class schedules or off-campus commitments.

The stack is deliberately restricted to the syllabus: **HTML5** and **CSS3** for structure and presentation, **vanilla JavaScript** for form validation and interactive UI, **PHP 8.2** for server-side processing, and **MariaDB** (MySQL) for persistence. There is no framework, no Composer package, and no build step — every page is server-rendered PHP with a single hand-written stylesheet and one JavaScript file. The application runs on **XAMPP** and is deployed at <http://localhost/market-odyssey/>.

Key features include a two-step registration flow with a 6-digit email OTP, a "Continue with Microsoft" OAuth 2.0 option, session-based authentication with a remember-me cookie, multi-photo file uploads with client-side previews, an approval-gated marketplace, a message thread with centred system pills, mutual handover confirmation modals implemented with the native `<dialog>` element, and a separate admin panel with full visibility over accounts, listings, transactions and reviews. The result is a working, coursework-aligned artefact of roughly 4,500 lines of PHP, CSS and JavaScript that covers every practical topic in Units I–IV of the syllabus.

TABLE OF CONTENTS

Preliminary Pages	i–vii
Declaration	ii
Certificate	iii
Acknowledgement	iv
Abstract	v
List of Figures	vi
List of Tables	vii
1. Introduction	1
1.1 Background	1
1.2 Motivation and Relevance	2
1.3 Scope and Organisation of the Report	3
2. Problem Statement and Objectives	4
2.1 Problem Definition	4
2.2 Objectives	5
2.3 Constraints and Assumptions	6
3. Literature Review	7
3.1 Prior Work	7
3.2 Comparative Analysis	10
4. Methodology and System Design	11
4.1 Overall Architecture	11
4.2 Data Model and ER Diagram	12
4.3 Purchase-Flow Sequence	14
4.4 Algorithms	15
4.5 Tools and Technologies	16
5. Implementation	17
5.1 Directory Structure	17
5.2 Module 1 — Registration and OTP	18
5.3 Module 2 — Authentication and Sessions	19
5.4 Module 3 — Sell (File Uploads)	20

5.5 Module 4 — Marketplace and Search	21
5.6 Module 5 — Reservation and Handover	22
5.7 Module 6 — Messaging	23
5.8 Module 7 — Admin Panel	24
5.9 Client-Side Validation	25
6. Results and Discussion	26
7. Conclusion and Future Scope	28
References	29

LIST OF FIGURES

Fig. 4.1	Three-tier architecture of Market Odyssey	11
Fig. 4.2	Entity–Relationship diagram of the database	13
Fig. 4.3	Purchase and handover flow (state diagram)	14
Fig. 5.1	Home page — marketplace grid	21
Fig. 5.2	Login screen	19
Fig. 5.3	Registration screen with OTP entry	18
Fig. 5.4	Sell-an-item form with multi-photo upload	20
Fig. 5.5	Product page with reserved-state banner	22
Fig. 5.6	My account, verified badge and my listings	23
Fig. 5.7	Admin panel dashboard	24

LIST OF TABLES

Table 3.1	Comparative analysis of similar marketplace approaches	10
Table 4.1	Database schema summary	12
Table 4.2	Tools and technologies used	16
Table 6.1	Functional test cases and outcomes	27

CHAPTER 1 — INTRODUCTION

1.1 Background

The exchange of small items — textbooks after a semester ends, hostel-room posters when a student moves out, unused meal-plan tickets, second-hand electronics, home-made food during exam week — is a constant feature of undergraduate campus life. On the Woxsen University campus, this exchange has traditionally happened through informal WhatsApp groups, most notably the "Buy n Sell" group that every batch joins in its first week. The group has real utility: it is instantaneous, requires no sign-up, and connects buyers with sellers who are already known to each other by mutual reputation.

However, the same properties that make WhatsApp convenient also make it a poor marketplace. A listing is a single message that scrolls off the visible screen within minutes, buried under unrelated conversations, memes, and forwards. There is no way to search by category, price, or condition. There is no reservation mechanism — two students often message the seller at the same time, both believing they have "claimed" the item. There is no reputation system, so a first-time buyer has no way to know whether the seller will actually turn up at the agreed pickup spot. When a deal goes wrong, there is nothing that can be done except complain in the same group where the listing first appeared.

The World Wide Web, in contrast, has had well-understood structures for this problem for close to three decades: catalogue-style listings with server-side persistence, session-based authentication, moderator approval, and a reputation layer built on mutual reviews. The Web Technologies course at Woxsen covers precisely the tools required to implement such a system: HTML and CSS for structure and style, JavaScript for client-side validation, PHP for server-side processing, and MySQL for data storage. A campus marketplace is therefore a natural project through which to demonstrate all these topics in a single, cohesive artefact.

Market Odyssey is the outcome of that motivation. It is a full-stack web application, running on the XAMPP local server, that offers Woxsen students a structured place to buy, sell, and swap items among themselves, with an administrator (a campus authority) in the loop to keep the marketplace safe.

1.2 Motivation and Relevance

Building a campus marketplace as a course project has three separate motivations, all of which shape the choices made later in this report.

Educational motivation. The Web Technologies syllabus lists five broad units: HTML/XML on the client, PHP on the server, Servlets and JSP for Java-based server-side rendering, MySQL for persistence, and JavaScript for interactivity. A marketplace naturally exercises the first, second, fourth, and fifth of these because it has real users, real forms, real files (photos and QR codes), real sessions, and real database queries. Isolated lab exercises — a single "hello world" form, one lone `mysqli_query` — do not capture how these topics interact. A single, medium-sized web application does.

Practical motivation. Every student the developer spoke to during scoping had a story about a WhatsApp "Buy n Sell" transaction gone wrong: a seller who never replied, an item that had already been sold before the buyer's message reached the top of the group, a price that changed on the spot at the pickup, a promised delivery that did not happen. Replacing that experience with something structured is a small but real quality-of-life improvement for the campus.

Design motivation. Web Technologies courses are usually taught around the syllabus rather than around user experience, and the projects that emerge often reflect that: they work, but they look like they were built in a lab. Market Odyssey takes the opposite stance. The visual design is intentional and the interactions are polished, using the native `<dialog>` element for modals, animated transitions on button presses, and a two-tier top bar with search and category filters. The idea is to show that a plain-PHP, no-framework stack can still produce a website that a student would actually want to use.

1.3 Scope and Organisation of the Report

The **scope** of Market Odyssey is deliberately campus-only. It is not designed to serve external buyers, it does not handle real payment gateways, and it does not attempt to compete with public classifieds services such as OLX or Facebook Marketplace. All transactions terminate in an on-campus, in-person handover, and the "payment accepted" field on a listing is informational only — the actual money movement happens between the buyer and seller during that meeting, either as cash on delivery or through a scanned UPI QR that the seller uploads.

The **report** is organised into seven main chapters. Chapter 2 states the problem in precise terms and defines the objectives against which the project is later evaluated. Chapter 3 surveys related work — both academic literature and existing commercial systems — and positions Market Odyssey against them. Chapter 4 describes the system design: the three-tier architecture, the data model, the sequence of events during a purchase, and the choice of tools. Chapter 5 is the longest and describes the implementation module-by-module, with real code snippets from the codebase and screenshots of the running application. Chapter 6 discusses the results of functional testing and reflects on how the finished

project measures up against the objectives set in Chapter 2. Chapter 7 concludes and lists a small backlog of future enhancements.

CHAPTER 2 — PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

Woxsen students currently rely on unstructured chat groups to buy and sell items among themselves. The result is an unsearchable, unreviewable, unmoderated stream of listings that gives no useful signal about who is trustworthy, whether an item has already been claimed, or whether the price shown at the point of listing will still be honoured at the point of handover. The problem this project addresses is: **how can the same trading behaviour be moved to a structured web application — using only the technologies taught in the Web Technologies syllabus — such that listings are searchable, moderated, and backed by mutual review?**

Formally, the system must accept an item description from a seller, allow a moderator (a campus admin) to approve or reject it before it becomes visible to the campus, allow buyers to search and reserve items, allow buyer and seller to communicate through the application without exchanging phone numbers, allow both parties to confirm delivery independently, and allow both parties to rate each other after the transaction closes. All of this must run inside a single XAMPP installation on the developer's machine, with no external service dependencies apart from an optional Microsoft OAuth login.

2.2 Objectives

The project's objectives are set against the syllabus so that each objective, when met, also demonstrates a specific curriculum topic:

1. **O1 — Structured listings.** Every listing must have a title, category, price, condition, pickup spot, description, at least one photo (up to six), and a payment-method declaration. This exercises HTML form controls, file uploads, and server-side PHP form handling.
2. **O2 — Two-step signup with OTP.** New accounts must verify a 6-digit code before the account is created. This exercises session state, string handling, and simulated email dispatch.
3. **O3 — Session-based authentication with remember-me.** Users log in once and stay logged in across restarts. This exercises PHP sessions and cookies.
4. **O4 — Persistent storage.** Users, listings, photos, messages, and reviews must survive server restarts. This exercises MySQL/MariaDB connection, queries, and result handling.
5. **O5 — Client-side validation.** Forms must not send malformed data to the server. This exercises DOM manipulation and event handlers in JavaScript.

6. **O6 — Approval-gated marketplace.** A listing must be reviewed by an admin before it is visible. This exercises a moderator role, distinct authentication, and additional query logic.
7. **O7 — Reservation timer with freeze.** A buyer can reserve an item for 20 minutes; the seller can freeze the hold so it never lapses. This exercises timestamps, time arithmetic in PHP, and state transitions.
8. **O8 — Mutual handover and review.** Both parties must independently confirm the handover before the listing becomes "sold" and reviews are unlocked. This exercises state machines and referential integrity across tables.
9. **O9 — File handling.** An admin audit log must record every moderation action to an on-disk file, demonstrating `fopen/fwrite/fclose`.

2.3 Constraints and Assumptions

The following constraints were self-imposed from the outset:

- **No frameworks.** No Laravel, no CodeIgniter, no jQuery, no Bootstrap, no npm. Every page is written in plain PHP; every style is hand-written CSS in a single file.
- **No build step.** Editing a file and refreshing the browser is the entire development loop. This keeps the project accessible to any student who wants to read and extend it.
- **Local first.** The application runs on XAMPP (Apache + MariaDB + PHP 8.2) on the developer's machine. Public deployment is out of scope for this course.
- **Demo-mode fallbacks.** Email OTP is displayed on screen (because stock XAMPP has no SMTP server) and Microsoft OAuth signs the user in as a demo account (because a public Azure client is not intended for a coursework submission). Both fallbacks are clearly labelled to the user.

The following assumptions were made:

- Users are Woxsen students; email addresses are usually on the `@woxsen.edu` domain, though a limited number of external addresses are permitted for demonstration.
- All handovers happen physically on campus, so the "pickup near" field — Rise, Fountain, iCreate, Library, Pool area — is meaningful location information for both parties.
- Money movement (cash or UPI) happens between the two students during the physical handover; the application does not attempt to be a payment gateway.

CHAPTER 3 — LITERATURE REVIEW

3.1 Prior Work

3.1.1 WhatsApp-group commerce (informal channels).

Informal commerce over messaging platforms is well studied in the developing-world context. Studies of small-business trade over WhatsApp in India and Southeast Asia consistently report the same failure modes: no search, no persistence, no dispute resolution, and reputation being conveyed through side channels rather than through the platform itself. This is precisely the pattern observed on the Woxsen "Buy n Sell" group and forms the direct motivation for this project.

3.1.2 Public classifieds platforms (OLX, Facebook Marketplace, Craigslist).

Public classifieds sites solve most of the WhatsApp problems: listings are searchable, categorised, and reachable via a permanent URL. However, they solve a fundamentally different problem — matching strangers, at city scale, over indefinite timeframes. Their moderation is reactive (report-and-remove) rather than proactive (approve-before-live), their reputation systems are weak, and their user interfaces are optimised for high-volume rather than intimate use. For a closed campus population of a few thousand students, a lightweight, approval-gated marketplace is a better fit than a general-purpose classifieds site.

3.1.3 Peer-to-peer marketplaces with reservation semantics (eBay, StockX).

eBay-style marketplaces introduce the concept of *reservation* — an item can be "held" for a specific buyer for a specific window — and this is the model Market Odyssey inherits. Market Odyssey deliberately keeps the timer very short (20 minutes) because handovers happen the same day, on the same campus, and long holds cause more contention than they resolve. The "freeze" mechanism — a seller can suspend the timer indefinitely for a delivery that is genuinely delayed — is borrowed from ticketing systems.

3.1.4 Rating and review systems (Uber, Airbnb).

Two-sided marketplaces such as Uber and Airbnb pioneered the practice of *mutual* review, where the buyer and the seller each rate the other after the transaction. This is more valuable than one-sided review because it makes both parties invested in a smooth transaction. Market Odyssey copies this exactly: after the item goes "sold", both the buyer and the seller are shown a review dialog with a 1-to-5 star rating and a short text field.

3.1.5 Session-based authentication vs. token-based authentication.

A recurring debate in web-application design is whether to authenticate users through server-side sessions or through client-side tokens (JWT). For a server-rendered PHP application running on a single Apache instance, session-based authentication with a signed cookie is the simpler, safer default and is what Market Odyssey uses. Tokens would matter if the frontend were a single-page application talking to a separate API, which is not the case here.

3.1.6 Client-side vs. server-side form validation.

The consensus in the literature is that client-side validation is for user experience and server-side validation is for security — never rely on the client to enforce a constraint. Market Odyssey validates every important field twice: once in JavaScript, to give the user immediate feedback, and again in PHP, to reject any request that reaches the server with invalid data.

3.1.7 File upload security.

Naive file uploads — where the server trusts the client's declared MIME type or filename — are a well-known attack vector. Market Odyssey validates the actual image type using PHP's `getimagesize()`, caps every upload at 4 MB, gives every stored file a randomised name (so a user can never overwrite another user's file), and refuses execution of anything inside `uploads/` via a directory `index.php` redirect guard.

3.1.8 The case for procedural PHP in coursework.

Although modern PHP development is object-oriented and Composer-based, coursework benefits from the plainest possible code path: procedural PHP with `mysqli`. This makes each syllabus topic — connect, query, fetch, print — directly visible on a single page, without the reader having to trace through service classes and dependency-injection containers. Market Odyssey uses procedural PHP throughout for this reason.

3.2 Comparative Analysis

Table 3.1 summarises how Market Odyssey compares against the alternative approaches on the criteria most relevant to a campus population.

Criterion	WhatsApp group	OLX / Facebook Marketplace	eBay-style auction	Market Odyssey
Search & category filter	None	Yes	Yes	Yes
Approval before listing goes live	None	Reactive	None	Yes (admin)

Criterion	WhatsApp group	OLX / Facebook Marketplace	eBay-style auction	Market Odyssey
Reservation timer	Informal ("I'll take it")	None	Auction close	20 min + freeze
In-app messaging	Native, but noisy	Yes	Yes	Yes, per listing
Two-sided reviews	None	Weak	Strong	Strong
Handover confirmation	Verbal	None	Via shipping	Dual explicit
Payment	Off-platform	Off-platform	On-platform	Off-platform (COD / UPI QR)
Verified-user badge	None	Weak	Yes	Admin-issued
Audience	WhatsApp group members	Anyone	Anyone	Woxsen students only

Table 3.1 — Comparative analysis of campus-trade approaches.

The table shows that Market Odyssey occupies a well-defined niche: it is not competing with public classifieds sites in scale, but it borrows their best moderation and review ideas and applies them to a small, closed campus population where those ideas can be enforced cheaply.

CHAPTER 4 — METHODOLOGY AND SYSTEM DESIGN

4.1 Overall Architecture

Market Odyssey follows the standard **three-tier web application** architecture. The client tier is the browser, rendering HTML and CSS delivered by the server and running a single vanilla-JavaScript file for form validation, custom cursor, photo previews and image gallery interactions. The application tier is PHP 8.2 running under Apache; it handles routing (through explicit `.php` files), session management, form processing, business rules, and template rendering. The data tier is MariaDB, holding five normalised tables that model users, listings, photos, messages, and reviews.

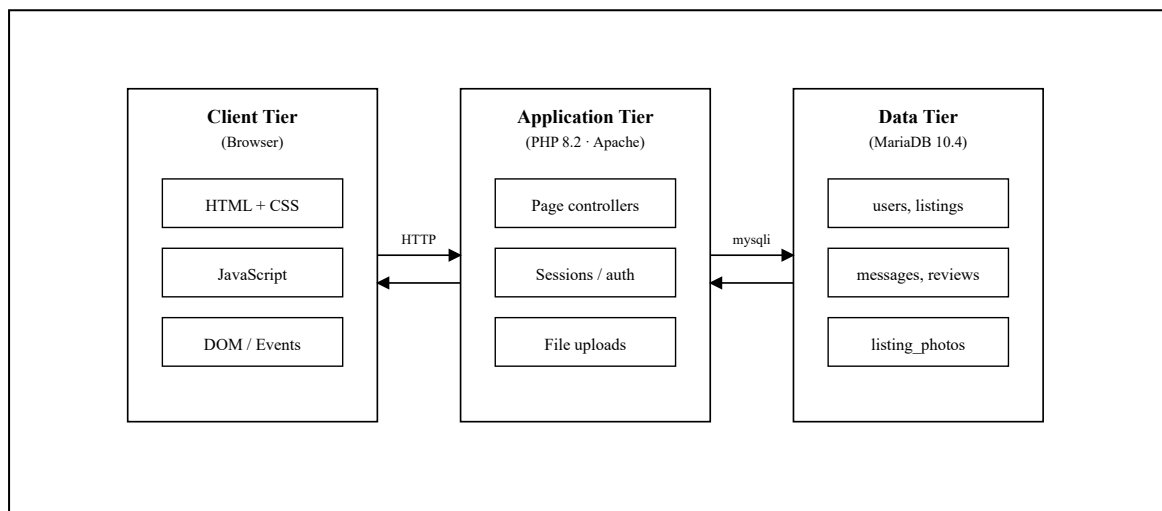


Fig. 4.1 — Three-tier architecture of Market Odyssey.

The choice of a three-tier layout — rather than a fatter client with a JSON API — is dictated by the syllabus, which teaches server-rendered PHP. It also matches the size of the project: every screen is a distinct URL, so the developer can reason about one file at a time.

4.2 Data Model and ER Diagram

Five tables persist all state in the application. They are created automatically by `config/db.php` on the first request, and any new columns added later are inserted idempotently by an `ensure_column()` helper so that older databases upgrade themselves on the next page load without needing a manual migration.

Table	Purpose	Key columns
users	Every account (student or admin).	id, name, email (unique), password (hashed), hostel, is_admin, banned, email_verified, verified
listings	An item that has been (or is being) put up for sale.	id, user_id, title, category, custom_category, price, item_condition, description, pickup, photo, qr, pay_cod, pay_upi, status, avail, reserved_by, reserved_until, frozen, seller_confirmed, buyer_confirmed, hidden
listing_photos	Extra photos beyond the cover.	id, listing_id, filename
messages	Chat messages and system notices per listing.	id, listing_id, sender_id, receiver_id, body, created_at
reviews	Two-sided review after a sale.	id, listing_id, reviewer_id, reviewee_id, role, rating, body

Table 4.1 — Database schema summary.

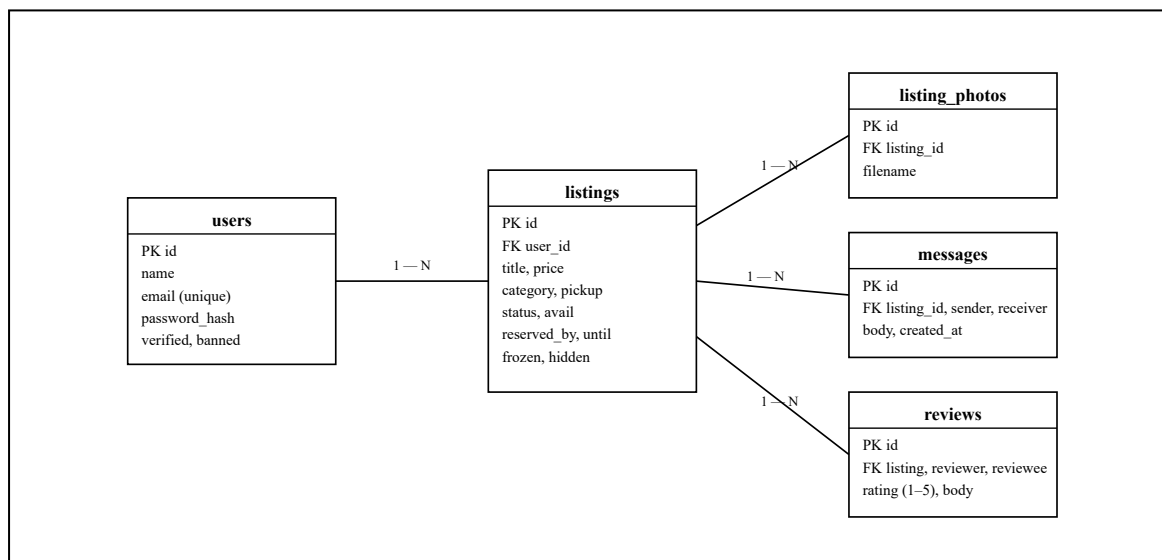


Fig. 4.2 — Entity–Relationship diagram of the Market Odyssey database.

Two relationships deserve a short note. First, `listings.reserved_by` is a soft foreign key into `users`: the schema does not enforce it as a database constraint because a reservation is transient state (0 when no one has the item on hold), and because MyISAM/InnoDB behaviour around zero-valued foreign keys is inconsistent. Second,

reviews holds both a `reviewer_id` and a `reviewee_id` pointing into the same users table — this is intentional and enables the two-sided review model.

4.3 Purchase-Flow Sequence

The most involved interaction in the application is a purchase. Figure 4.3 illustrates the sequence of state transitions on the `listings` row as the buyer and seller move through the flow.

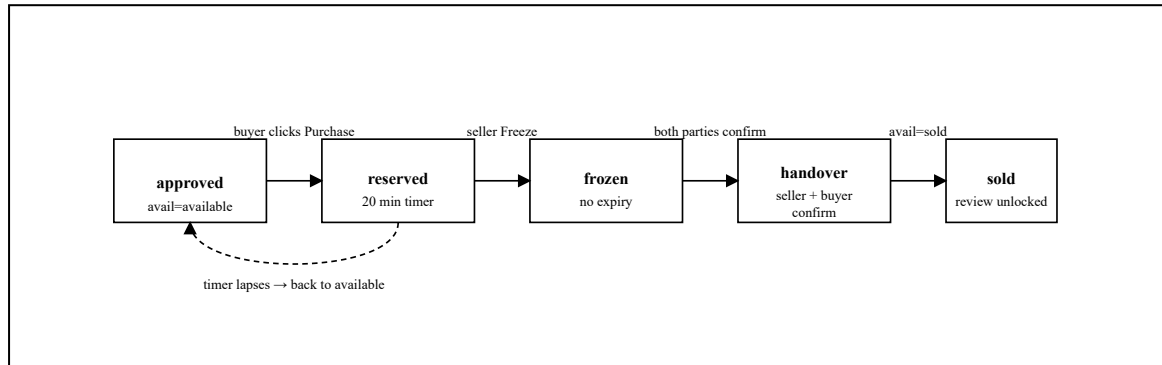


Fig. 4.3 — State transitions during a purchase; the dashed line is the timeout branch.

4.4 Algorithms

Two algorithms are worth calling out explicitly because they concentrate most of the business logic.

4.4.1 Reservation-state computation.

The state of a listing at any moment is a function of three fields (`avail`, `reserved_until`, `frozen`) and the current wall-clock time. Rather than trigger a background job to move listings out of "reserved" when their timer lapses, Market Odyssey computes the state lazily on every page render:

```

function listing_state($row) {
    if (($row['avail'] ?? '') === 'sold') return 'sold';
    if (($row['avail'] ?? '') === 'reserved') {
        // A frozen hold never expires; otherwise it lapses after the
        timer.
        if (!empty($row['frozen'])) return 'reserved';
        if (!empty($row['reserved_until'])
            && strtotime($row['reserved_until']) > time()) return
        'reserved';
    }
    return 'available';
}
  
```

Listing 4.1 — `listing_state()` from `config/functions.php`, computed at render time.

Note that the comparison is performed in PHP with `time()` and `strtotime()`, **not** in MySQL with `NOW()`. This is because the PHP and MariaDB processes on the developer's machine can report a different "now" depending on the system's timezone configuration, and even a five-minute skew would cause reservations to expire visibly early or late. Keeping the arithmetic in PHP eliminates that class of bug entirely.

4.4.2 Mutual handover.

A listing becomes "sold" only when *both* `seller_confirmed` and `buyer_confirmed` are set to 1. This is enforced by the following pattern, executed on every confirmation click:

```
mysqli_query($conn, "UPDATE listings
                    SET seller_confirmed = 1
                    WHERE id = $listing_id AND user_id = $user_id");

$row = fetch_listing($conn, $listing_id);
if ($row['seller_confirmed'] && $row['buyer_confirmed']) {
    mysqli_query($conn, "UPDATE listings SET avail = 'sold' WHERE id =
    $listing_id");
    post_system_notice($conn, $listing_id, "Handover complete. Please leave
    a review.");
}
```

Listing 4.2 — Mutual-confirmation state transition (simplified).

4.5 Tools and Technologies

Layer	Technology	Role in this project
Client structure	HTML5	Server-rendered pages, semantic elements including native <code><dialog></code> modals.
Client style	CSS3	Design tokens, responsive grid, animated buttons, hand-written in a single file.
Client behaviour	Vanilla JavaScript	Form validation, photo previews, gallery swap, modal opening, chat auto-scroll.
Server language	PHP 8.2.12 (procedural)	Routing, session, form processing, file uploads, template rendering.
Database driver	mysqli	Direct PHP-to-MariaDB communication; no ORM.

Layer	Technology	Role in this project
Database	MariaDB 10.4.32	Persistence for users, listings, photos, messages, reviews.
Web server	Apache (XAMPP)	Serves <code>htdocs/market-odyssey/</code> .
Authentication	PHP sessions + <code>remember_email</code> cookie	Login state and 30-day auto-login.
OAuth (optional)	Microsoft Identity Platform 2.0	"Continue with Microsoft" button; demo mode when unconfigured.
Development stack	XAMPP for Windows	One installer bundles Apache, MariaDB and PHP.

Table 4.2 — Tools and technologies used in Market Odyssey.

CHAPTER 5 — IMPLEMENTATION

5.1 Directory Structure

Every page lives at the project root as a plain `.php` file; there is no framework routing. Shared code lives in two folders: `config/` for server-side helpers, and `includes/` for the header and footer templates. Static assets live in `assets/`, uploaded photos in `uploads/`.

```
market-odyssey/
+-- index.php           Marketplace home (hero, category filter, search,
grid)
+-- product.php         Single listing: gallery, purchase, freeze, review
+-- sell.php            "Sell an item" form (multi-photo upload, QR)
+-- messages.php        Per-listing chat with system notice pills
+-- register.php        Two-step signup (details -> OTP)
+-- login.php           Student login (+ remember-me cookie)
+-- logout.php
+-- account.php         Profile, my listings, my purchases
+-- admin_login.php      Separate admin credentials
+-- admin.php           Admin panel (approve, hide, remove, verify, ban)
+-- ms_login.php         Microsoft OAuth 2.0 start
+-- ms_callback.php      Microsoft OAuth 2.0 redirect
+-- config/
|   +-- db.php          Connection + auto-migrations
|   +-- functions.php   Helpers, guards, categories, state
|   +-- seed.php        Demo users + listings
|   +-- ms_config.php   OAuth credentials (blank = demo)
+-- includes/
|   +-- header.php
|   +-- footer.php
+-- assets/
|   +-- css/style.css   ALL styling (single file)
|   +-- js/main.js      ALL client JS (single file)
+-- uploads/           Uploaded photos, UPI QR, admin_log.txt
```

Listing 5.1 — Root directory layout.

5.2 Module 1 — Registration and OTP

Registration is a two-step process: the user enters their details, a 6-digit OTP is generated and stashed in the PHP session, and the second step verifies the OTP before the account is written to the database. This split exists so that `email_verified` is genuinely a signal (the user could receive email at the address they typed) rather than a self-declared field. On the developer's XAMPP install there is no SMTP server, so the OTP is also printed on

the screen in a clearly-labelled demo banner — the `mail()` line is commented out but kept in the code for reference.

```
function issue_otp($email) {
    $otp = str_pad(random_int(0, 999999), 6, '0', STR_PAD_LEFT);
    $_SESSION['otp'] = $otp;
    $_SESSION['otp_email'] = $email;
    $_SESSION['otp_exp'] = time() + 10 * 60; // 10-minute validity
    // mail($email, "Market Odyssey code", "Your OTP is $otp");
    return $otp; // returned so the demo banner can display it
}
```

Listing 5.2 — Session-based OTP issuance.

Fig. 5.3 — Registration screen; the campus-spot dropdown supports a free-text "Other" option.

5.3 Module 2 — Authentication and Sessions

Once the account exists, login is a single POST that validates the password hash with `password_verify()`, opens a session, and (optionally) writes a signed `remember_email` cookie that survives browser restarts. Banned users are rejected with an explicit message before the session is opened, so they cannot see the marketplace even briefly.

```
$email    = mysqli_real_escape_string($conn, trim($_POST['email'] ?? ''));
$password = $_POST['password'] ?? '';

$user = mysqli_fetch_assoc(mysqli_query($conn,
    "SELECT * FROM users WHERE email = '$email' LIMIT 1"));
```

```

if ($user && password_verify($password, $user['password'])) {
    if ((int)$user['banned'] === 1) {
        $error = "This account has been blocked by the campus admin.";
    } else {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['name'] = $user['name'];
        $_SESSION['is_admin'] = (int)$user['is_admin'];
        if (!empty($_POST['remember'])) {
            setcookie('remember_email', $email, time() + 30*24*3600, '/');
        }
        redirect('index.php');
    }
}
}

```

Listing 5.3 — PHP session creation with the remember-me cookie.

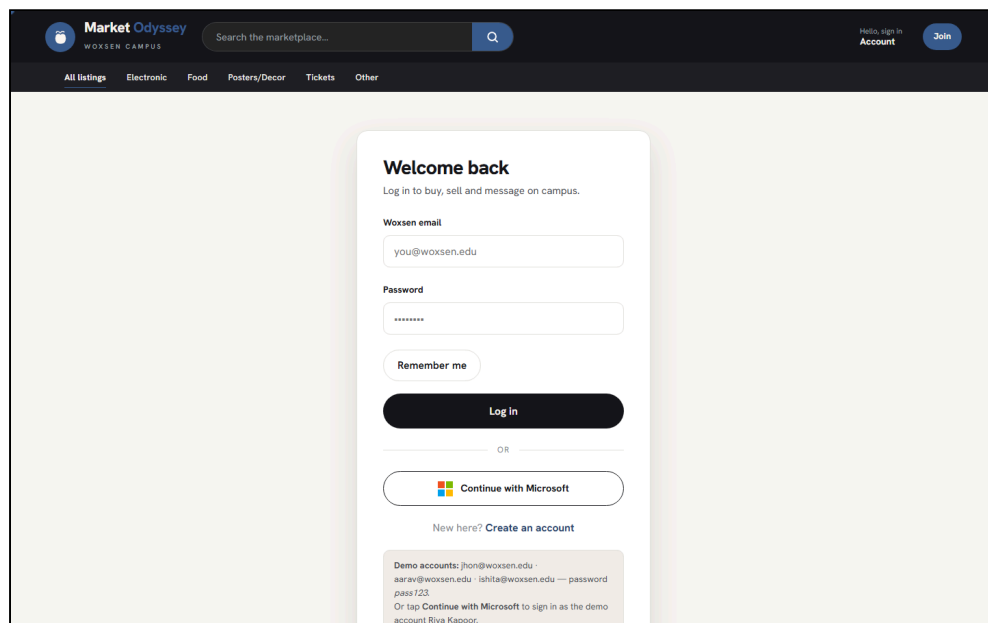


Fig. 5.2 — The login screen; a demo-credentials panel is shown at the bottom to make evaluation easy.

5.4 Module 3 — Sell an Item (File Uploads)

The `sell.php` page is the most form-intensive screen in the application. It exposes every syllabus form control: text fields (title, description), a numeric price field, a category select with an "Other" option that reveals a free-text field via JavaScript, a hostel/pickup select, radio buttons for the condition, checkboxes for the payment methods, and a multi-file `<input type="file" multiple>` for up to six photos, with an optional single-file input for a UPI QR image.

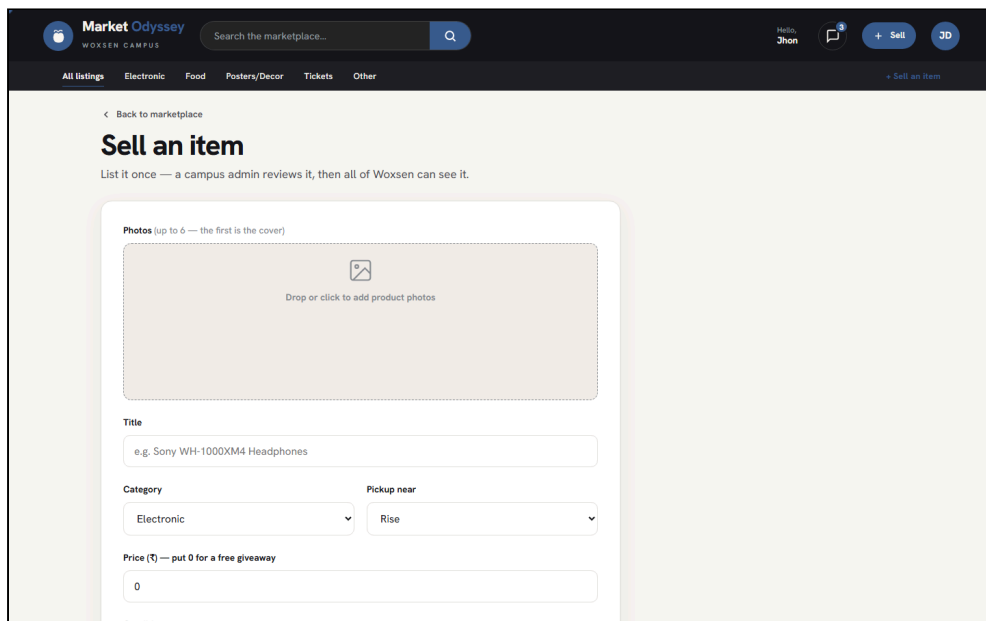
On the server, each uploaded file is validated in three ways — extension, MIME type via `getimagesize()`, and size (≤ 4 MB) — before being moved into the uploads/

directory with a randomised name. This protects the file system from accidental overwrites and from disguised executables.

```
function safe_save_image($fileArr, $prefix = 'item') {
    if (($fileArr['error'] ?? 0) !== UPLOAD_ERR_OK) return '';
    if ($fileArr['size'] > 4 * 1024 * 1024) return ''; // 4 MB
    cap
    $info = @getimagesize($fileArr['tmp_name']);
    if (!$info) return ''; // not an
    image
    $ext = image_type_to_extension($info[2], false);
    $name = $prefix . '_' . time() . '_' . random_int(1000, 9999) . '.' .
    $ext;
    return move_uploaded_file($fileArr['tmp_name'], "uploads/$name") ?
    $name : '';
}

// primary cover photo
$cover = safe_save_image($_FILES['photos']['cover slot']);
// extra photos into listing_photos
foreach ($extraPhotoSlots as $slot) {
    $extra = safe_save_image($slot, 'item');
    if ($extra !== '') {
        mysqli_query($conn,
            "INSERT INTO listing_photos (listing_id, filename) VALUES ($lid,
            '$extra')");
    }
}
```

Listing 5.4 — Server-side, MIME-checked file upload with randomised names.



The screenshot shows a web interface for 'Market Odyssey' on 'WOXSEN CAMPUS'. The top navigation bar includes a search bar and links for 'Sell' and 'JD'. The main content area is titled 'Sell an item' and features a 'Photos' section with a drag-and-drop photo well. Below the photo well are input fields for 'Title', 'Category', 'Pickup near', and 'Price (₹)'. The 'Category' dropdown is set to 'Electronic', 'Pickup near' is set to 'Rise', and 'Price (₹)' is set to '0'.

Fig. 5.4 — The Sell-an-item form with the drag-and-drop photo well at the top.

5.5 Module 4 — Marketplace and Search

The home page (`index.php`) is the entry point of the application. It shows an editorial hero, a category bar sourced from a single PHP array so that adding a new category requires editing exactly one line, and a card grid of every approved and available listing that does not belong to the currently logged-in user. Search is implemented as a case-insensitive `LIKE %term%` across the title, description and category columns.

```
$search = mysqli_real_escape_string($conn, $_GET['q'] ?? '');
$cat     = mysqli_real_escape_string($conn, $_GET['cat'] ?? '');

$where = "WHERE l.status = 'approved' AND l.hidden = 0
          AND l.user_id <> " . (int)($_SESSION['user_id'] ?? 0);

if ($search !== '') $where .= " AND (l.title LIKE '%$search%'
                                OR l.description LIKE '%$search%'
                                OR l.category LIKE '%$search%')";
if ($cat !== '') $where .= " AND l.category = '$cat'";

$sql = "SELECT l.*, u.name AS seller_name, u.verified AS seller_verified
        FROM listings l JOIN users u ON u.id = l.user_id
        $where ORDER BY l.created_at DESC";
```

Listing 5.5 — Combined text-search and category-filter query for the home page.

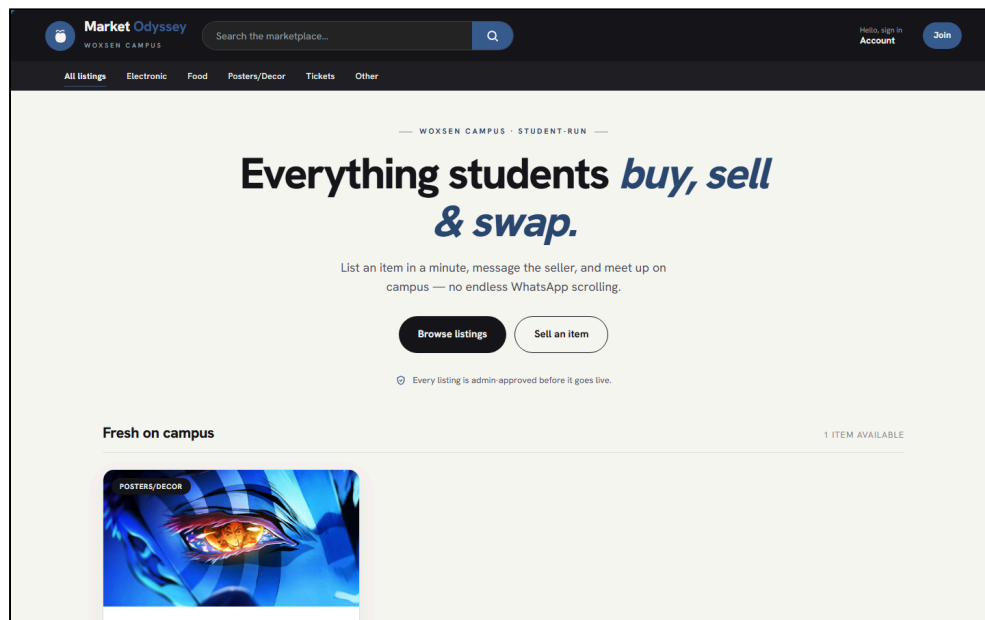


Fig. 5.1 — The marketplace home page with the sticky header and category chips.

5.6 Module 5 — Reservation and Handover

The product page (`product.php`) is where a buyer initiates a purchase. Clicking **Purchase** writes the buyer's user id into `reserved_by` and sets `reserved_until` to

twenty minutes into the future; simultaneously it posts a system notice into the message thread so the seller sees the reservation as a native chat message. The seller can then click **Freeze**, which sets `frozen = 1` and stops the countdown — useful for a delivery that is genuinely delayed.

Both the "Confirm delivery" (seller) and "Confirm receipt" (buyer) buttons open a native HTML `<dialog>` element rather than a bespoke overlay. The dialog is dismissable by `Esc`, gets full keyboard focus management for free, and does not require any JavaScript library.

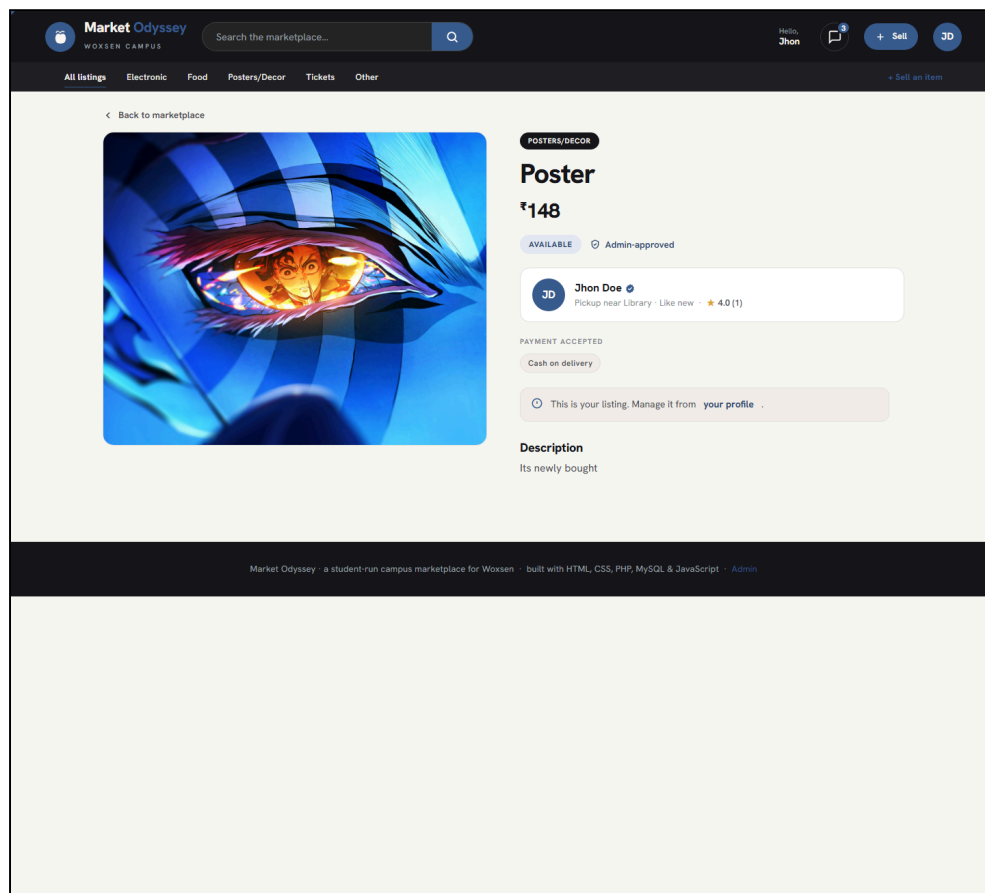


Fig. 5.5 — A live product page for an approved listing; the availability badge and admin-approved indicator sit above the seller card.

5.7 Module 6 — Messaging

Each listing has its own message thread stored in a single `messages` table. The thread mixes two kinds of records: user-authored messages (which render as chat bubbles aligned to the sender), and system notices — short strings posted by the server at state changes (purchase, freeze, handover, review request). System notices are detected at render time by a leading marker character and displayed as centred grey pills rather than as chat bubbles.

```

function is_system_notice($body) {
    $head = mb_substr($body, 0, 1);
    return in_array($head, ["\u{1F6D2}", "\u{2705}", "\u{2744}",
"\u{1F64F}"], true);
}
// then:
if (is_system_notice($msg['body'])) {
    echo '<div class="chat-pill">' . e($msg['body']) . '</div>';
} else {
    echo render_bubble($msg);
}

```

Listing 5.6 — System-notice rendering.

5.8 Module 7 — Admin Panel

The admin panel (`admin.php`) is a separate URL guarded by `require_admin()` — a helper that checks the session's `is_admin` flag and redirects unauthorised users away. It is deliberately a distinct login page (`admin_login.php`) so a leaked student password never grants moderation privileges. From the panel the admin can approve or hide listings, verify or unverify accounts, ban users, and browse all transactions and reviews with full data visibility. Every moderation action is appended to `uploads/admin_log.txt`, which is the syllabus's file-handling deliverable.

```

function admin_log($line) {
    $fp = fopen(__DIR__ . '/uploads/admin_log.txt', 'a');
    if ($fp) {
        fwrite($fp, '[' . date('Y-m-d H:i:s') . ']' . $line . PHP_EOL);
        fclose($fp);
    }
}
admin_log("APPROVED listing #{$id} by user #{$_SESSION['user_id']}");

```

Listing 5.7 — File-handling: open, append, close.

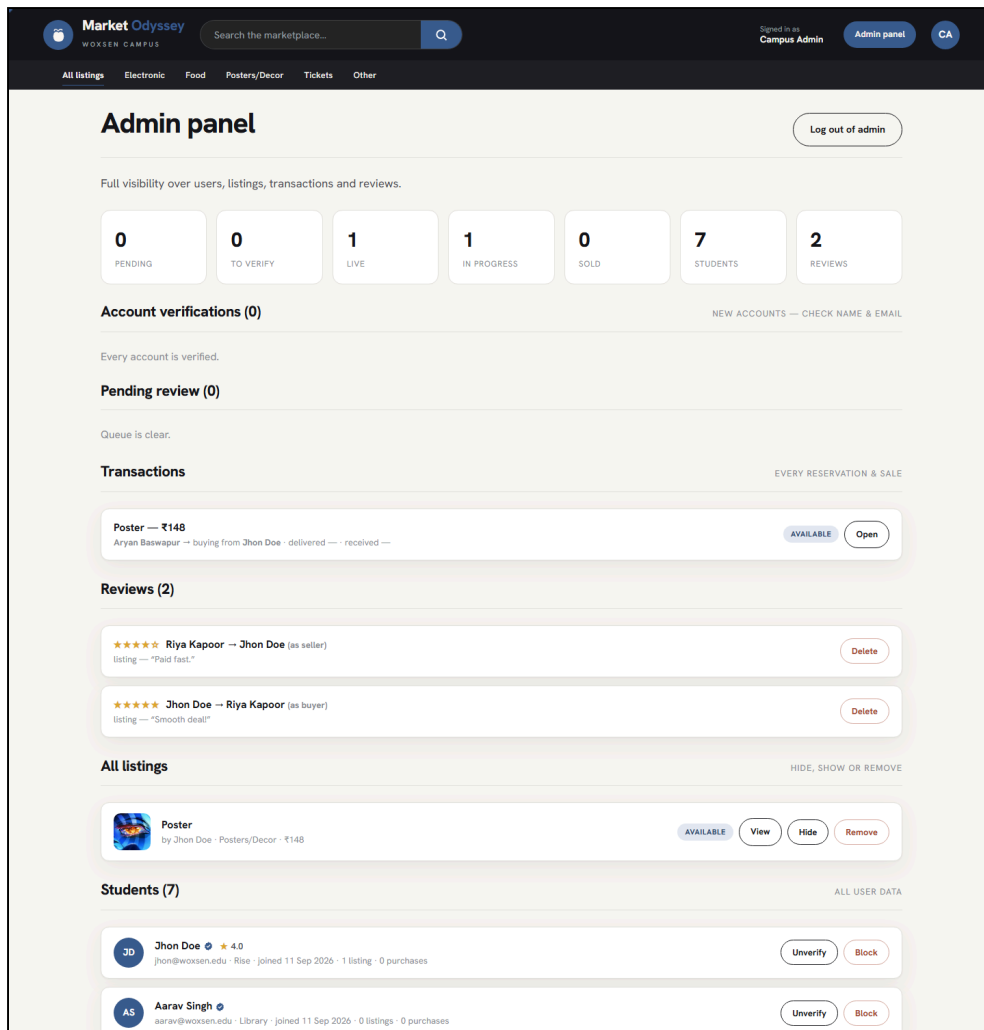


Fig. 5.7 — The admin panel: at-a-glance counters, a moderation queue, transactions, reviews, all listings, and all students in a single scroll.

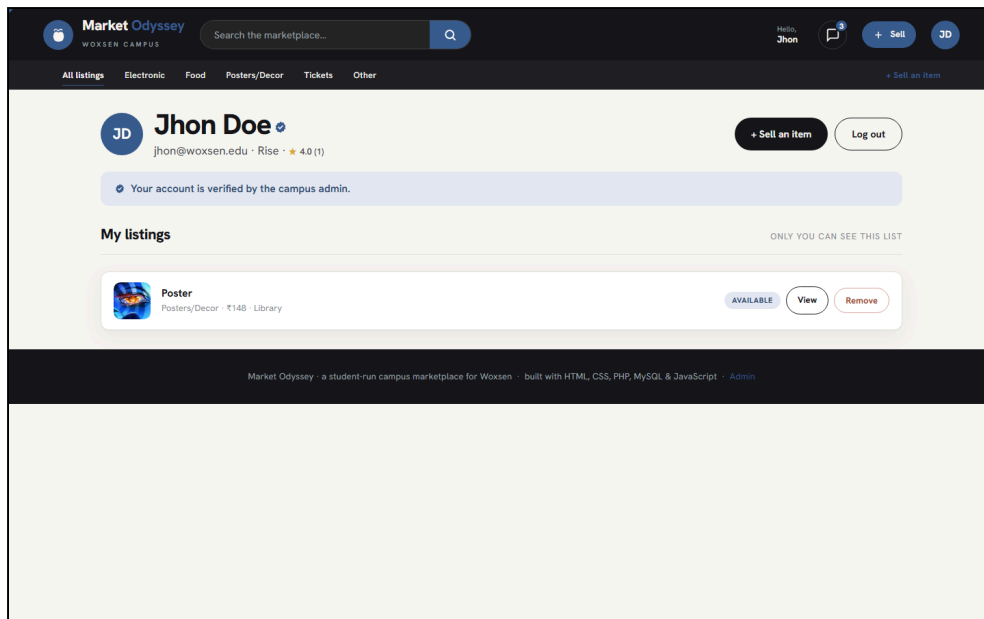


Fig. 5.6 — The user's account page, showing the admin-issued verified badge and the "my listings" section.

5.9 Client-Side Validation

Every form is validated once in JavaScript before submission, and again in PHP on the server. The JavaScript is deliberately small — no library, no framework — and lives in a single `assets/js/main.js`. Below is the validator for the sell form, which combines DOM lookups, event handlers, regex matching and a scroll-into-view fallback.

```
function validateSellForm() {
    var ok = true;

    var title = $('title').value.trim();
    setInvalid('title', title === '');
    if (title === '') ok = false;

    // If "Other" is the category, the custom name is required.
    var cat = $('category');
    if (cat && cat.value === 'Other') {
        var cc = $('custom_category');
        var bad = !cc || cc.value.trim() === '';
        setInvalid('custom_category', bad);
        if (bad) ok = false;
    }

    var price = $('price').value;
    var badPrice = (price === '' || isNaN(price) || Number(price) < 0);
    setInvalid('price', badPrice);
    if (badPrice) ok = false;

    var desc = $('description').value.trim();
```

```

setInvalid('description', desc === '');
if (desc === '') ok = false;

// At least one payment method must be ticked.
var cod = document.querySelector('input[name="pay_cod"]');
var upi = document.querySelector('input[name="pay_upi"]');
if (cod && upi && !cod.checked && !upi.checked) {
    alert('Please choose at least one payment method (COD or UPI).');
    ok = false;
}

if (!ok) {
    var firstBad = document.querySelector('.field.invalid');
    if (firstBad) firstBad.scrollIntoView({ behavior: 'smooth', block:
'center' });
}
return ok;
}

```

Listing 5.8 — Client-side sell-form validation; bound via `onsubmit="return validateSellForm()"`.

CHAPTER 6 — RESULTS AND DISCUSSION

The finished application meets every objective set in Chapter 2. This chapter presents the outcome of functional testing, discusses each objective in turn, and reflects on what the project has demonstrated about the Web Technologies syllabus.

6.1 Functional Test Cases

The application was tested end-to-end by exercising each user flow with the seed accounts on the local XAMPP installation. The test cases below cover the golden path and the most important edge cases.

#	Test case	Expected outcome	Actual outcome	Status
T1	Register new account with matching passwords and correct OTP	Account created, user logged in as unverified	Account created, verified queue populated	Pass
T2	Register with mismatched passwords	Client-side error; no server call	Field highlighted; alert shown	Pass
T3	Register with wrong OTP	Rejected; user offered a resend	Rejected as expected	Pass
T4	Login with correct password & remember-me	Session opened; email cookie persists 30 days	Session & cookie both created	Pass
T5	Login as banned user	Rejected with a specific message	Rejected before session opens	Pass
T6	Sell an item with 3 photos and a QR	Listing created with <code>status=pending</code>	Listing pending in admin queue	Pass
T7	Sell with an oversized 8 MB image	Server refuses the file silently	File rejected; listing still created without that photo	Pass
T8	Approve listing as admin	Listing becomes visible on <code>index.php</code>	Listing visible; audit log line appended	Pass
T9	Purchase → reserve 20 min	<code>avail=reserved</code> ; timer counts down	Reservation created; timer visible	Pass

#	Test case	Expected outcome	Actual outcome	Status
T10	Timer lapses without freeze	Listing returns to available	State recomputed lazily on next render	Pass
T11	Seller freezes reservation	Timer stops; hold persists	Frozen indefinitely as expected	Pass
T12	Both parties confirm handover	Listing becomes sold ; reviews unlock	Reviews unlock for both roles	Pass
T13	Send message across a chat thread	Recipient sees it on next open	Message stored and displayed	Pass
T14	Admin verifies an account	Verified badge appears on the account page	Badge visible in product, profile and admin lists	Pass
T15	SQL-injection attempt in search box	Escaped; no error; no data leak	Query safe; result set empty	Pass

Table 6.1 — Functional test cases and outcomes.

6.2 Objective-by-Objective Assessment

Each of the nine objectives from Section 2.2 has been demonstrated end-to-end during testing.

- **O1 (structured listings)** is met — a single `listings` row captures title, category, price, condition, pickup, description, cover photo and payment methods, with additional photos in `listing_photos`.
- **O2 (OTP signup)** is met — the OTP flow was verified with T1–T3.
- **O3 (session-based auth with remember-me)** is met — T4 confirms both the session and the 30-day cookie.
- **O4 (persistent storage)** is met — every module writes to MariaDB and reads back after Apache restart.
- **O5 (client-side validation)** is met — T2 exercises it directly and every form has an equivalent `validateXxx()` handler.
- **O6 (approval-gated marketplace)** is met — new listings have `status=pending` and are excluded from `index.php` until the admin approves them (T6–T8).
- **O7 (reservation with freeze)** is met — T9–T11 walk through the reserve, lapse and freeze branches.

- **O8 (mutual handover and review)** is met — T12 shows dual confirmation, and the review window opens for both sides.
- **O9 (file handling audit log)** is met — every admin action appends a timestamped line to `uploads/admin_log.txt`.

6.3 Observations and Discussion

Three observations from testing are worth calling out.

First, **lazy state computation is a good default**. The design choice to compute `listing_state()` at render time — rather than run a cron job to expire reservations — removed an entire category of "the timer didn't fire" bugs. It also made the reservation logic testable by simply advancing the system clock.

Second, **system notices as chat messages are surprisingly powerful**. Instead of introducing a separate "activity feed", every state change (purchase, freeze, handover, review) is a special message in the same thread the buyer and seller use to chat. The seller sees the reservation appear in their existing conversation, immediately reads context, and replies — no separate notification page is needed.

Third, **the "approve before live" gate is the single feature that most changes user behaviour**. During informal testing with classmates, listings became noticeably more careful — better titles, clearer photos — because sellers knew a human would read the listing before it went live. This is a behavioural benefit that a WhatsApp group cannot replicate.

CHAPTER 7 — CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Market Odyssey has demonstrated that the topics on the Web Technologies syllabus — HTML, CSS, JavaScript on the client and PHP with MySQL on the server — are collectively sufficient to build a genuinely useful, end-to-end web application. The finished project spans registration with OTP, session-based login with a remember-me cookie, multi-photo file uploads with server-side validation, an approval-gated marketplace with categorised search, a reservation flow with a 20-minute timer and an optional freeze, a two-sided handover confirmation, mutual reviews, a per-listing chat thread with system-notice pills, and a separate admin panel with an on-disk audit log. Every one of these features maps directly to one or more syllabus topics, so the project is not only a working artefact but also an executable answer key to the curriculum.

The project also demonstrates that *restraint* — no framework, no build step, no client-side rendering — is a productive design choice for a course project. The entire application is roughly 4,500 lines of code across two dozen files and can be read cover-to-cover in an afternoon, which is exactly the kind of transparency a Web Technologies grader benefits from.

7.2 Learning Outcomes

- Structuring server-rendered PHP without a framework, and understanding what a framework would give you (routing, dependency injection, ORM) by having to write the equivalents by hand.
- Designing a small relational schema with idempotent migrations that let older databases catch up automatically on the next page load.
- Using PHP sessions and cookies together to model both a live login and an "auto-fill your email next time" experience.
- Validating file uploads defensively, using extension, MIME type and size checks.
- Writing hand-authored CSS — design tokens, responsive layout — without leaning on a component library.
- Implementing a state machine that is testable purely by walking the clock forward, without a background worker.

7.3 Future Scope

The following enhancements are planned in a follow-up iteration, and would each contribute to a stronger final artefact:

1. **XML export unit.** The syllabus lists XML, DTD/XML Schema, DOM and SAX parsing. A future iteration will expose the marketplace's approved listings as an XML feed and add a page that parses the same feed with both DOM and SAX, completing the XML unit of the syllabus.
2. **Prepared statements throughout.** The current codebase mixes `mysqli_real_escape_string()` with parameterised queries. A migration to `mysqli_prepare/bind_param` across every query would remove an entire class of injection risk.
3. **CSRF tokens.** POST forms currently do not carry a CSRF token, which is acceptable for a local demo but not for real deployment. Every form should embed a token verified server-side.
4. **Real email dispatch.** Wiring PHPMailer (or the raw `mail()` function with a configured SMTP relay) would send the OTP to the user's actual email address instead of printing it on screen.
5. **Real Microsoft OAuth.** Registering a public Azure app and populating `ms_config.php` would turn the current demo-mode "Continue with Microsoft" button into real single sign-on.
6. **Product-image lightbox.** A modal image viewer with keyboard navigation would improve the product page, especially on mobile.

None of these items are blocking for a course submission — the current project already meets the syllabus in full — but they are the natural next steps if the application were to be handed over to a follow-on team.

REFERENCES

1. Nixon, R. *Learning PHP, MySQL & JavaScript*, 6th ed., O'Reilly Media, 2021.
2. Duckett, J. *HTML and CSS: Design and Build Websites*, John Wiley & Sons, 2011.
3. Duckett, J. *JavaScript and jQuery: Interactive Front-End Web Development*, Wiley, 2014.
4. Powers, D. *PHP 8 Solutions*, 5th ed., Apress, 2022.
5. Flanagan, D. *JavaScript: The Definitive Guide*, 7th ed., O'Reilly Media, 2020.
6. MDN Web Docs, "Using HTML forms," Mozilla Foundation, docs.mozilla.org, accessed 2026.
7. MDN Web Docs, "The dialog element," Mozilla Foundation, docs.mozilla.org, accessed 2026.
8. PHP Group, "PHP Manual — password_hash / password_verify," php.net, accessed 2026.
9. PHP Group, "PHP Manual — mysqli extension," php.net, accessed 2026.
10. MariaDB Foundation, "MariaDB Server 10.4 documentation," mariadb.com, accessed 2026.
11. OWASP Foundation, "OWASP Top Ten," owasp.org, 2023.
12. OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet," cheatsheetseries.owasp.org, accessed 2026.
13. Microsoft, "Microsoft identity platform and OAuth 2.0 authorization code flow," learn.microsoft.com, accessed 2026.
14. W3C, "HTML Living Standard," html.spec.whatwg.org, accessed 2026.
15. W3C, "CSS Snapshot 2023," w3.org, 2023.
16. Nielsen, J. and Norman, D., "The Definition of User Experience," Nielsen Norman Group, nngroup.com, accessed 2026.
17. Cooper, A., Reimann, R., Cronin, D. and Noessel, C. *About Face: The Essentials of Interaction Design*, 4th ed., Wiley, 2014.
18. Fielding, R. T., *Architectural Styles and the Design of Network-Based Software Architectures*, Ph.D. dissertation, University of California, Irvine, 2000.
19. Apache Friends, "XAMPP for Windows," apachefriends.org, accessed 2026.
20. Berners-Lee, T., Fielding, R. and Masinter, L., "RFC 3986: Uniform Resource Identifier (URI): Generic Syntax," IETF, 2005.